



Security Audit

SilverSwap (LSW)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	18
Audit Resources	18
Dependencies	18
Severity Definitions	19
Status Definitions	20
Audit Findings	21
Centralisation	36
Conclusion	37
Our Methodology	38
Disclaimers	40
About Hashlock	41

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Silverswap team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

SilverSwap.io is a decentralized exchange (DEX) that enables users to trade and swap cryptocurrencies in a secure, non-custodial manner. The platform is built to facilitate decentralized trading without the involvement of intermediaries, offering users full control over their digital assets. By operating on blockchain technology, SilverSwap ensures transparency and security in transactions, while promoting a peer-to-peer trading model. Users can participate in liquidity pools, where they provide liquidity in exchange for potential rewards, helping to maintain the platform's fluidity and enabling seamless swaps of various digital assets.

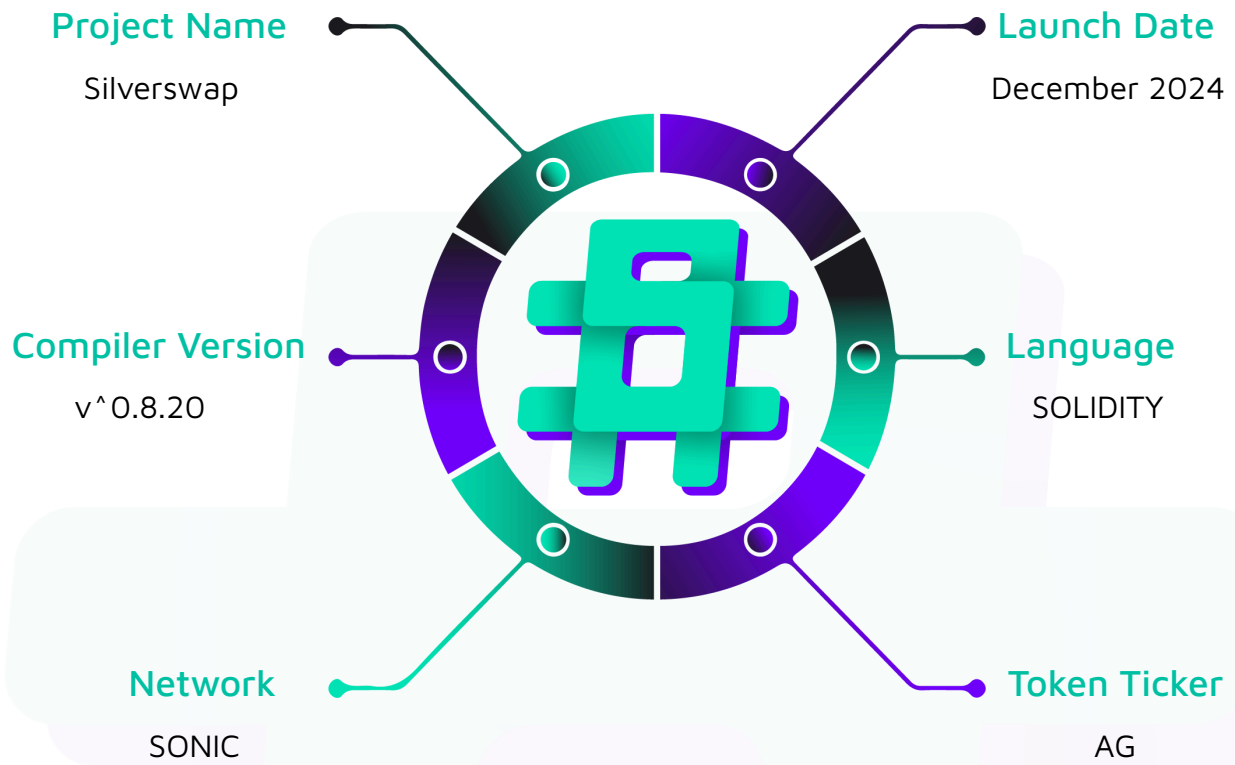
Project Name: Silverswap

Compiler Version: ^0.8.20

Website: <https://silverswap.io/>

Logo:



Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the Silverswap project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Silverswap Smart Contracts
Platform	Ethereum / Solidity
Audit Date	July, 2025
Contract 1	PoolLocker.sol
Contract 2	GaugeManager.sol
Contract 3	SilverLswManager.sol,
Contract 4	SilverVoteLSW.sol
Contract 5	GaugeTaskExecutor.sol
Contract 6	ContractPermissionManager.sol
Contract 7	TimelockProtection.sol
Contract 8	TimelockAdmin.sol
Contract 9	TimelockMain.sol
Contract 10	SilverStakeToken.sol
Contract 11	SilverStake.sol
Contract 12	SilverValidatorAuth.sol
Audited GitHub Commit Hash	11c11cbceac86a66ba8ec266623240660fa47e1a
Fix Review GitHub Commit Hash	497c7737a820448c59cbbc59cc03782cbd7b8550

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved.

Hashlock found:

3 High severity vulnerabilities

2 Medium severity vulnerabilities

1 Low severity vulnerability

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
SilverStake.sol Allows users to: - Stake native tokens (S) to receive liquid AGS tokens - Unstake AGS tokens to receive native tokens after a 14-day delay - Withdraw unstaked native tokens after the delay period - Transfer AGS tokens with vesting benefits - Benefit from anti-frontrunning vesting mechanism (10-100% rewards based on time held) Allows SilverLswManager to: - Compound validator rewards back into staking - Update exchange rates based on accumulated rewards - Claim validator staking rewards Allows governance to: - Update validator ID with 21-day timelock protection - Confirm validator transfers after delay period - Set vesting parameters and minimum stake amounts	Contract achieves this functionality.

- Emergency fund withdrawal capabilities	
SilverStakeToken.sol - Allows the protocol to: - Mint AGS tokens when users stake native tokens - Burn AGS tokens when users unstake - Track token transfers to maintain vesting information - Allows users to: - Transfer AGS tokens between accounts - Receive proportional vesting benefits when receiving transfers - Use AGS tokens for governance voting and gauge participation	Contract achieves this functionality.
PoolLocker.sol Allows users to: - Create and lock LP positions for potential gauge creation - Extend lock periods for existing positions - Unlock positions after lock period expires - Withdraw trading fees from locked positions - Increase lock time for better gauge consideration Allows gauge manager to: - Accept positions for gauge creation - Refuse positions and return them to users	Contract achieves this functionality.

- Unlock positions in batches during system operations Allows admins to: - Purge old pending gauge requests (after 2 weeks) - Remove positions from pending lists - Emergency position management	
GaugeManager.sol Allows users to: - Add liquidity to active gauges for voting power - Remove liquidity from gauges - Participate in gauge voting through AGS deposits - Earn rewards based on gauge voting participation Allows admins to: - Initialize new gauges from locked positions - Refuse gauge creation requests - Emergency fund withdrawal capabilities Allows SilverLswManager to: - Add liquidity to wrapped native token pools - Remove 5% liquidity from pools during reward distribution - Unlock positions during system operations - Manage automated reward distribution	Contract achieves this functionality.

processes	
TimelockMain.sol Allows timelock role holders to: - Schedule critical protocol changes with a 21-day delay - Execute scheduled transactions after the delay period - Cancel scheduled transactions if needed - Modify minimum delay within bounds (21-30 days) Allows admin to: - Activate timelock protection permanently - Grant and revoke timelock execution roles Provides system-wide protection for: - Validator changes and core infrastructure modifications - Protocol parameter updates affecting economics - Emergency governance actions requiring extended review	Contract achieves this functionality.
TimelockAdmin.sol Allows timelock role holders to: - Schedule administrative changes with a 1-3 day delay - Execute routine maintenance operations - Cancel scheduled administrative	Contract achieves this functionality.

transactions Modify the delay within administrative bounds Allows admin to: - Activate administrative timelock protection - Manage administrative role assignments Provides protection for: - Treasury fund movements - Gauge management operations - Routine parameter adjustments - Administrative maintenance tasks	
TimelockProtections.sol Provides a modifier framework to: - Restrict critical functions to timelock contracts only - Fallback to owner control when timelocks are inactive - Enforce proper governance flow for protected operations - Separate main timelock vs admin timelock requirements Allows contracts to: - Implement graduated protection levels - Maintain operational flexibility before timelock activation - Ensure proper separation of concerns	Contract achieves this functionality.

between critical and routine operations	
SilverLSWManager.sol Allows the automated system to: - Orchestrate 14-day LSW system cycles with Gelato automation - Claim validator rewards and split them 50/50 (compounding vs gauge rewards) - Calculate gauge voting power percentages and reward distributions - Swap 50% of rewards to project tokens and reinject as liquidity - Break 5% of liquidity from pools and distribute to voters - Lock/unlock system during epoch operations to prevent interference Allows gauge operations to: - Create and manage Gelato tasks for automated gauge operations - Execute gauge-specific reward calculations and distributions - Handle liquidity reinjection through automated swapping - Manage gauge executors for decentralized task execution Allows admins to: - Start/stop automated system cycles with configurable timing - Emergency fund withdrawal and system	Contract achieves this functionality.

reset capabilities Update critical contract addresses with timelock protection - Configure reward distribution scripts and parameters	
SilverVoteLSW.sol Allows users to: - Vote on active gauges using their AGS voting power - Change votes between gauges (only one vote per user) - Participate in gauge governance through token-weighted voting - Influence reward distribution through a democratic voting process Allows SilverLswManager to: - Clear all voting information at the end of each epoch cycle - Access voter lists and voting power calculations - Reset system state for the next voting period - Query gauge voting status and participation data Provides system-wide functionality for: - Democratic gauge selection and reward allocation	Contract achieves this functionality.

- Time-based vote validity (votes expire after system cycles) - Voting power calculation based on AGS deposits in gauges - Transparent voter tracking and gauge participation metrics	
GaugeTaskExecutor.sol Allows Gelato automation to: - Execute gauge-specific snapshot operations for data collection - Perform automated liquidity breaking and compounding tasks - Operate independently per gauge for decentralized execution - Interface with SilverLswManager for coordinated operations Provides gauge-specific services: - Dedicated executor per gauge for isolated task management - Two-type task execution (snapshot and liquidity operations) - Secure execution is limited to authorized Gelato callers - Immutable gauge association for consistent operations Enables system architecture to: - Scale task execution across multiple gauges simultaneously	Contract achieves this functionality.

- Maintain the separation of concerns between gauge operations

Support parallel processing of gauge-related automation

- Ensure reliable task execution through dedicated executors

Code Quality

This audit scope involves the smart contracts of the Silverswap project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Silverswap project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] PoolLocker#mintAndLockPosition - Multiple users locking for the same position will result in a loss of fees

Description

There is a time gap between users locking their position via `mintAndLockPosition` and gauge initialization. However, if multiple users lock their position for the same pair, it will result in them paying the fees and locking a position that will only be freed until `purgePendingGauges` is called.

Vulnerability Details

Consider that User1 locks their position for 6 months via `mintAndLockPosition`. Here's how the entire process of minting and locking looks like.

One, some require checks if the system is not locked, `address(0)` validation, minimum amount provided. Two, transfer project and silverstake tokens, followed by transfer of 2000 project tokens and 1000 silverstake tokens to `GaugeManager`. Three, checking if gauge is active and creating a position and updating state variables via `lockPosition`, where the `tokenId` is pushed to `pendingGaugesTokenId`.

```
function mintAndLockPosition(
    uint256 lockedMonths,
    address projectToken,
    uint256 projectTokenAmount,
    uint256 silverStakeAmount
) public checkLockMonths(lockedMonths) {
    require(!gaugeManager.isLswLocked(), "Lsw system is locked");
    require(projectToken != address(0), "Invalid project token");
    require(projectToken != address(silverStakeToken), "Project token cannot
be SilverStake");
    require(projectTokenAmount > 2000 && silverStakeAmount > 1000, "Amounts
```

```

too low");

        uint256 balanceSilverStakeTokenBefore =
silverStakeToken.balanceOf(address(this));

        TransferHelper.safeTransferFrom(projectToken, msg.sender, address(this),
projectTokenAmount);

        TransferHelper.safeTransferFrom(address(silverStakeToken), msg.sender,
address(this), silverStakeAmount);

        projectTokenAmount -= 2000;
        silverStakeAmount -= 1000;

        TransferHelper.safeTransfer(projectToken, address(gaugeManager), 2000);
        TransferHelper.safeTransfer(address(silverStakeToken),
address(gaugeManager), 1000);

        TransferHelper.safeApprove(projectToken, address(positionManager),
projectTokenAmount);

        TransferHelper.safeApprove(address(silverStakeToken),
address(positionManager), silverStakeAmount);

        (address token0, address token1) = projectToken <
address(silverStakeToken) ?
            (projectToken, address(silverStakeToken)) :
            (address(silverStakeToken), projectToken);
        (uint256 amount0Desired, uint256 amount1Desired) = projectToken <
address(silverStakeToken) ?
            (projectTokenAmount, silverStakeAmount) : (silverStakeAmount,
projectTokenAmount);

        address pool = IAlgebraFactory(positionManager.factory()).getPair(token0,
token1);

        require(pool != address(0), "Pool not found");
        require(!gaugeManager.isGaugeActive(pool), "Gauge already active for this
projectToken");

        (uint256 tokenId, uint128 liquidity, uint256 amount0, uint256 amount1) =

```

```

positionManager.mint(
    INonfungiblePositionManager.MintParams({
        token0: token0,
        token1: token1,
        tickLower: -887220,
        tickUpper: 887220,
        amount0Desired: amount0Desired,
        amount1Desired: amount1Desired,
        amount0Min: 0,
        amount1Min: 0,
        recipient: address(this),
        deadline: block.timestamp + 1 hours
    })
);

uint256 realSilverStakeAmount = silverStakeAmount -
(silverStakeToken.balanceOf(address(this)) - balanceSilverStakeTokenBefore);
TransferHelper.safeTransfer(address(silverStakeToken), msg.sender,
silverStakeAmount - realSilverStakeAmount);

lockPosition(tokenId, token0, token1, msg.sender, realSilverStakeAmount,
lockedMonths, pool);

emit PositionMinted(positionByTokenId[tokenId].lockedPositionId, tokenId,
liquidity, amount0, amount1);
}

function lockPosition(uint256 _tokenId, address token0, address token1, address
_owner, uint256 silverStakeAmount, uint256 lockedMonths, address pool) private {
    require(!positionByTokenId[_tokenId].locked, "Position already locked");
    require(positionManagerERC721.ownerOf(_tokenId) == address(this),
"Position not transferred to contract");

    positionByTokenId[_tokenId] = PositionInfo({
        lockedPositionId: _incrementLockedPositionId(),
        tokenId: _tokenId,
        owner: _owner,
        token0: token0,

```

```

        token1: token1,
        silverStakeAmount: silverStakeAmount,
        locked: true,
        lockedMonths: lockedMonths,
        creationTime: block.timestamp,
        lockedUntil: block.timestamp + (lockedMonths * 30 days),
        gaugeCreated: false,
        pool: pool
    });
    _increaseLockedAgs(silverStakeAmount);
    tokenIdByLockedPositionId[positionByTokenId[_tokenId].lockedPositionId] =
_tokenId;

    pendingGaugesTokenId.push(_tokenId);
    userPositions[_owner].push(_tokenId);

    emit PositionLocked(positionByTokenId[_tokenId].lockedPositionId,
_tokenId, _owner);
}

```

As we can see, there is no check to stop multiple users from locking and mint for the same positions. Subsequent user's position will be pushed in `pendingGaugesTokenId` array.

Now, as soon as the gauge is initiated by the admin, all the other similar positions become useless. Even though their positions can be released as soon as `purgePendingGauges` is called by the owner but even in that process, the fee transfer to the gauge manager will not be refunded.

Impact

Users will lose their tokens to lock for the same position.

Recommendation

Add a check in `mintAndLockPosition` to check if there is a `tokenId` already present for the position that an incoming user is locking for.

Status

Resolved

[H-02] TimelockProtection#requireTimelockMain, requireTimelockAdmin - Time delay is not enforced, causing instantaneous executions.

Description

The timelock system implements a dual-layer protection mechanism with `TimelockMain` (21-day delay) and `TimelockAdmin` (1-3 day delay) contracts designed to secure critical protocol operations. However, the timelock contracts fail to enforce their intended time delays, allowing `TIMELOCK_ROLE` to execute scheduled transactions immediately without waiting for the mandatory delay period.

Vulnerability Details

The vulnerability exists in both `TimelockMain.sol` and `TimelockAdmin.sol` contracts due to the incomplete implementation of the delay enforcement mechanism:

```
function schedule(...) external onlyRole(TIMELOCK_ROLE) whenActivated {
    bytes32 txHash = keccak256(abi.encode(target, value, data, predecessor, salt));
    require(!_queuedTransactions[txHash], "Transaction already queued");

    _queuedTransactions[txHash] = true; //@audit Only boolean flag, no timestamp stored
    emit TransactionScheduled(txHash, target, value, data, predecessor, _minDelay);
}
```

```
function execute(...) external payable onlyRole(TIMELOCK_ROLE) nonReentrant
whenActivated {
    bytes32 txHash = keccak256(abi.encode(target, value, data, predecessor, salt));
    require(_queuedTransactions[txHash], "Transaction not queued");
    _queuedTransactions[txHash] = false;

    //@audit No verification that minimum delay has elapsed
    (bool success, ) = target.call{value: value}(data);
}
```

```
require(success, "Transaction failed");  
}
```

This means that `timelock` admin can instantaneously execute all the critical calls, which completely undermines the purpose of `timelock`. The vulnerability can also be confirmed as an unintended design through comments :

```
@dev Can only be called after timelock activation and after the delay period
```

Impact

Instantaneous execution by admin.

Recommendation

Enforce minimum delay when execution is scheduled.

Status

Resolved

[H-03] SilverStake#confirmValidatorTransfer - Accounting mismatch will result in an incorrect rate of one native token.

Description

The validator transfer mechanism fails to account for slashing penalties that occur during the 14-day withdrawal delay period. When `confirmValidatorTransfer()` is executed, slashing losses are not deducted from `totalStakedNative`, creating a permanent accounting mismatch where the contract's internal accounting overstates the actual staked funds.

Vulnerability Details

The vulnerability stems from the fact that `confirmValidatorTransfer` simply re-delegates whatever amount is received after withdrawal, without checking if it matches the expected amount or updating the accounting accordingly.

Looking at `sfc.withdraw` in `SFC.sol`

```
function _withdraw(address delegator, uint256 toValidatorID, uint256 wrID, address payable receiver) private {
    WithdrawalRequest memory request =
    getWithdrawalRequest[delegator][toValidatorID][wrID];

    // ... time delay checks ...

    uint256 amount = getWithdrawalRequest[delegator][toValidatorID][wrID].amount;

    //@audit Slashing check happens AT WITHDRAWAL TIME, not delegation time
    bool isCheater = isSlashed(toValidatorID);
    uint256 penalty = _getSlashingPenalty(amount, isCheater,
    slashingRefundRatio[toValidatorID]);

    delete getWithdrawalRequest[delegator][toValidatorID][wrID];

    if (amount <= penalty) {
        revert StakeIsFullySlashed(); // You could lose everything!
    }
}
```

```

    //@audit You only get: amount - penalty
    (bool sent, ) = receiver.call{value: amount - penalty}("");
    _burnNativeTokens(penalty); // Penalty is burned/lost forever
}

```

We can see how slashing penalties applies during withdrawal and not while undelegating.

This design is intentional to prevent delegator's escape punishment by quickly undelegating when they see the validator's misbehavior and a shared responsibility of loss if validator misbehaves.

As soon as `confirmValidatorTransfer` is called, it will withdraw and redelegate to the new validator.

```

function confirmValidatorTransfer() external requireTimelockMain {
    require(validatorTransfer.validatorID != 0, "No validator transfer");
    require(block.timestamp >= validatorTransfer.timestamp, "Not yet");

    uint256 balanceBefore = address(this).balance;
    sfc.withdraw(stakeSettings.validatorID, validatorTransfer.wrID);
    uint256 nativeAmount = address(this).balance - balanceBefore; //@audit -
    does not decrease totalStakedNative in case of slashing.
    stakeSettings.validatorID = validatorTransfer.validatorID;

    sfc.delegate{value: nativeAmount}(stakeSettings.validatorID);

    validatorTransfer.validatorID = 0;
    validatorTransfer.wrID = 0;
    validatorTransfer.amount = 0;
    validatorTransfer.timestamp = 0;
    validatorTransfer.transferInitiated = false;

    emit ValidatorTransferConfirmed(validatorTransfer.validatorID,
    validatorTransfer.wrID, validatorTransfer.amount, validatorTransfer.timestamp);
}

```

This can create an accounting mismatch where `totalStakedNative` will be inflated than the actual value and will directly impact `updateRateForOneToken` since it is using `totalStakedNative`.

```
function updateRateForOneToken() public onlySilverLswManager {
    uint256 totalMintedAGS = silverStakeTokenERC20.totalSupply();
    if (totalMintedAGS == 0) return;

    uint256 newRate = (totalStakedNative * 1e18) / totalMintedAGS;

    if (newRate != stakeSettings.rateForOneNative) {
        if (newRate < stakeSettings.rateForOneNative) {
            uint256 minAllowedRate = (stakeSettings.rateForOneNative * 90) / 100;
            if (newRate < minAllowedRate) {
                newRate = minAllowedRate;
            }
        }
        stakeSettings.rateForOneNative = newRate;
        emit RateUpdated(stakeSettings.rateForOneNative);
    }
}
```

Impact

Inflated rate of one native token.

Recommendation

Adjust for slashing losses during confirmation

Status

Resolved

Medium

[M-01] SilverStake#compounding Compounding during validator transfer can result in stuck funds

Description

The `compounding()` function lacks protection against validator transfers in progress, allowing funds to be delegated to the old validator during the transfer period. These compounded funds become permanently stuck because the transfer completion only withdraws the original undelegated amount, leaving compounded funds on the abandoned validator.

Vulnerability Details

Consider a scenario where a validator transfer is initiated. As we can see, during validator initiation via `updateValidatorID()`, undelegation happens, but `stakeSettings.validatorId` still points to the previous validator.

```
function compounding(uint256 wrappedNativeAmount) public onlySilverLswManager {
    uint256 balanceBefore = address(this).balance;
    wrappedNativeToken.withdraw(wrappedNativeAmount);
    uint256 nativeAmount = address(this).balance - balanceBefore;

    sfc.delegate{value: nativeAmount}(stakeSettings.validatorID);

    totalStakedNative += nativeAmount;

    emit Compounded(nativeAmount);
}
```

Additionally, an analysis of the `sfc.sol` contract shows that the undelegation process does not restrict users from delegating again to the same validator.

Re-delegation to a previously undelegated validator is possible because undelegation doesn't create any restrictions on future delegations - it only reduces your current active stake balance to zero (or whatever amount you undelegated). The SFC system

only checks two things when you try to delegate: does the validator exist, and is the validator still active (not deactivated for misbehavior). It doesn't care about your delegation history or whether you have pending withdrawal requests with that validator.

```
function _rawUndelegate(
    address delegator,
    uint256 toValidatorID,
    uint256 amount,
    bool strict,
    bool forceful,
    bool checkDelegatedStake
) internal {
    getStake[delegator][toValidatorID] -= amount;
    getValidator[toValidatorID].receivedStake =
getValidator[toValidatorID].receivedStake - amount;
    totalStake = totalStake - amount;
    if (getValidator[toValidatorID].status == OK_STATUS) {
        totalActiveStake = totalActiveStake - amount;
    }

    uint256 selfStakeAfterwards = getSelfStake(toValidatorID);
    if (selfStakeAfterwards != 0 && getValidator[toValidatorID].status ==
OK_STATUS) {
        if (!(selfStakeAfterwards >= c.minSelfStake())) {
            if (forceful) {
                revert InsufficientSelfStake();
            } else {
                _setValidatorDeactivated(toValidatorID, WITHDRAWN_BIT);
            }
        }
        if (checkDelegatedStake && !_checkDelegatedStakeLimit(toValidatorID)) {
            revert ValidatorDelegationLimitExceeded();
        }
    } else {
        _setValidatorDeactivated(toValidatorID, WITHDRAWN_BIT);
    }
}
```

```
    _notifyStakeSubscriber(delegator, getValidator[toValidatorID].auth, strict);  
}
```

This means that all compounding will be delegated to the previous validator instead.

Impact

Loss of compounding funds if it happens during the undelegation process. Severity is medium since the function is manager restricted.

Recommendation

Do not allow compounding if the validator transfer is initiated.

Status

Resolved

[M-02] PoolLocker, GaugeManager#mintAndLockPosition, addLiquidityInGauge, addLiquidityInWrappedPool, removeLiquidityFromWrappedPool, _createEmptyPosition -Lack of slippage protection leads to loss of funds.

Description

All liquidity operations across PoolLocker and GaugeManager contracts use zero slippage protection (amount0Min: 0, amount1Min: 0), making them vulnerable to MEV attacks, sandwich attacks, and unfavorable price movements that can result in significant value loss for users and the protocol.

Vulnerability Details

Here are all the instances where slippage protection is not implemented.

```
function mintAndLockPosition(...) {
    (uint256 tokenId, uint128 liquidity, uint256 amount0, uint256 amount1) =
    positionManager.mint(
        INonfungiblePositionManager.MintParams({
            // ...
            amount0Min: 0, // No slippage protection
            amount1Min: 0, // No slippage protection
            deadline: block.timestamp + 1 hours
        })
    );
}
```

```
function addLiquidityInGauge(...) {
    (uint128 liquidityAdded, uint256 amount0, uint256 amount1) =
    positionManager.increaseLiquidity(
        INonfungiblePositionManager.IncreaseLiquidityParams({
            // ...
            amount0Min: 0, // No slippage protection
            amount1Min: 0, // No slippage protection
        })
    );
}
```

```

function removeLiquidityFromGauge(...) {
    (uint256 amount0, uint256 amount1) = positionManager.decreaseLiquidity(
        INonfungiblePositionManager.DecreaseLiquidityParams({
            // ...
            amount0Min: 0, // No slippage protection
            amount1Min: 0, // No slippage protection
        })
    );
}

function addLiquidityInWrappedPool(...) {
    // Same issue - amount0Min: 0, amount1Min: 0
}

function removeLiquidityFromWrappedPool(...) {
    // Same issue - amount0Min: 0, amount1Min: 0
}

function _createEmptyPosition(...) {
    // Same issue - amount0Min: 0, amount1Min: 0
}

```

Impact

This could result in the user getting lp tokens for a skewed ratio.

Recommendation

Consider adding slippage protection.

Status

Resolved

Low

[L-01] TimeLockAdmin - Invalid minimum delay in TimeLockAdmin.sol

Description

The TimeLockAdmin.sol incorrectly used `MAX_DELAY` instead of `MIN_DELAY` in the constructor.

```
constructor(  
    address multisig  
) {  
    _minDelay = MAX_DELAY; //@audit adding max delay in min delay  
  
    _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);  
    _grantRole(TIMELock_ROLE, multisig);  
}
```

Recommendation

Consider using the minimum delay variable instead.

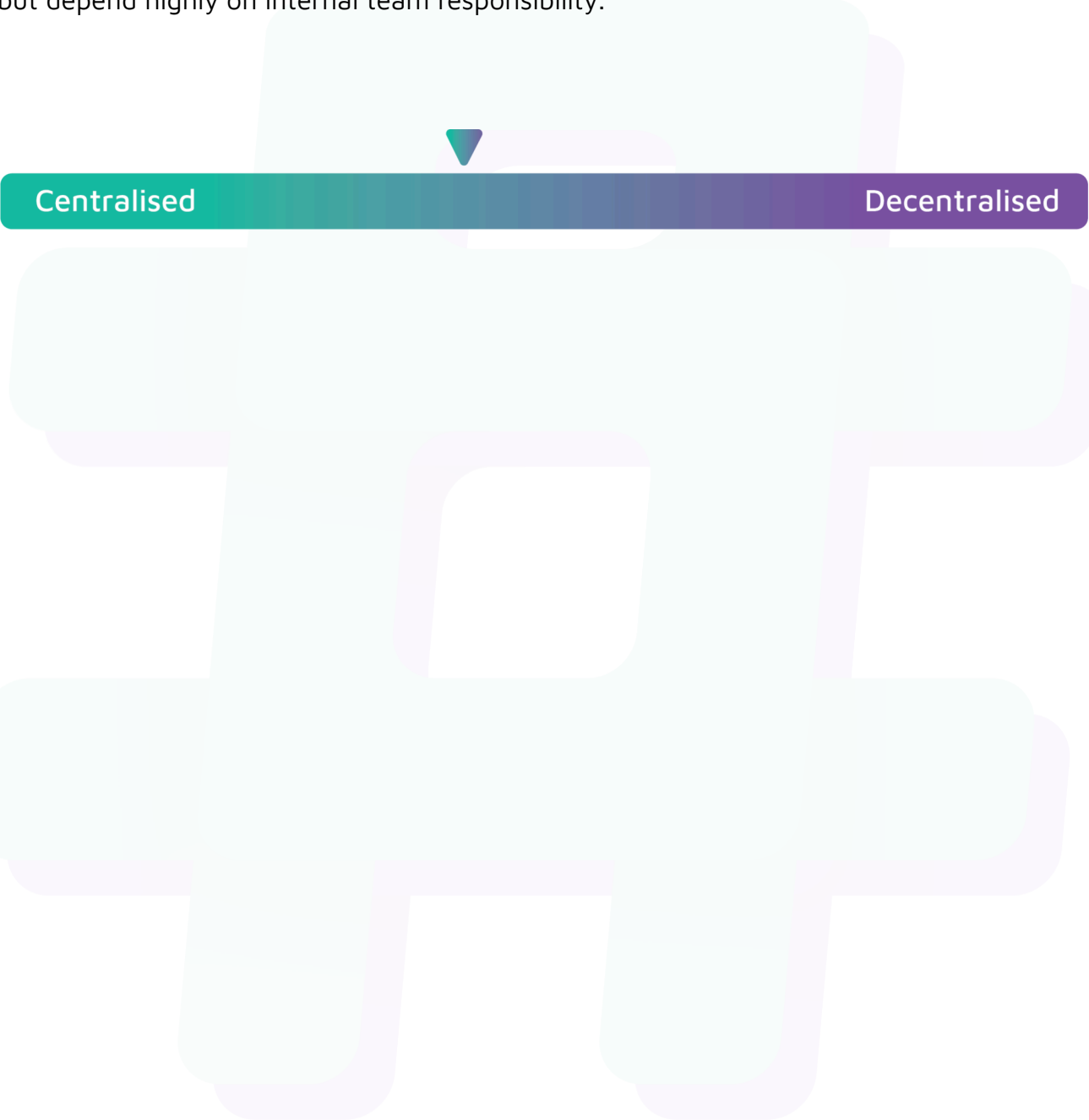
Status

Resolved

Centralisation

The Silverswap project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Silverswap project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.